

하네스 엔지니어링 (HARNESS ENGINEERING)



Agent = Model + Harness — 모델 너머의 게임

Constrain

Inform

Verify

Correct

AI 에이전트 시대의 새로운 상식

모델 선택보다 하네스 설계가 성과를 더 크게 좌우한다



LangChain TerminalBench 2.0 — Harness v1 vs v2 실측 결과

순위 상승

30위 → Top 5

점수 상승

52.8% → 66.5%

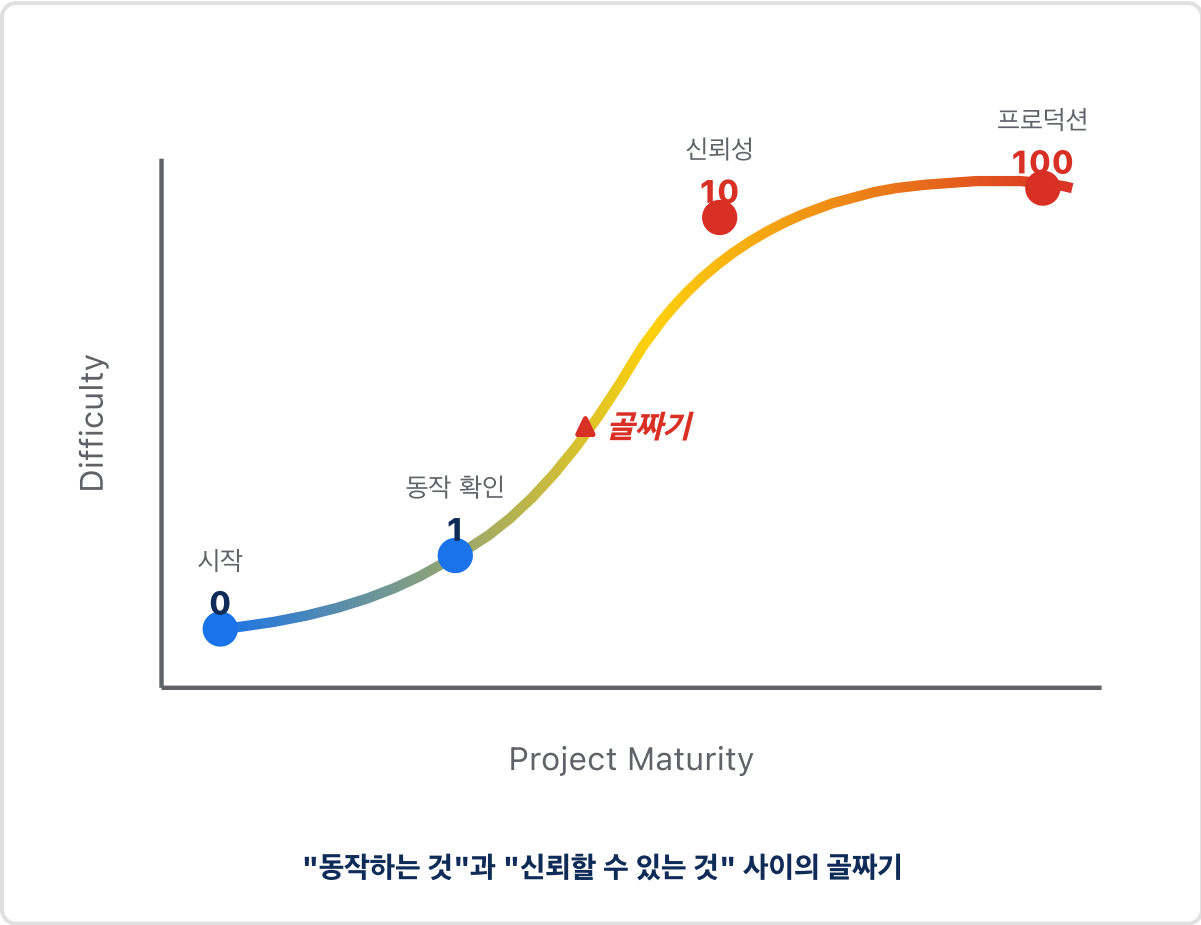
변수

Same Model, Different Harness

같은 **GPT-5.2-Codex** — 하네스만 교체

바이브코딩의 골짜기

0→1은 압도적이지만 1→10·100으로 갈 때 벽에 부딪힌다



왜 벽에 부딪히는가?

- 1 AI 생성 코드의 약 **45%**에서 보안 취약점 발견
- 2 같은 프롬프트 3번 → **3개의 다른 코드** (확률적 출력)
- 3 운영·감사·복구 메커니즘이 **전혀 없음**
- 4 Gartner: 에이전틱 AI 프로젝트의 **40%**가 **2027년 중단** 예측

40% **2027년까지 중단**
에이전틱 AI 프로젝트 중단 예측
— Gartner, 2025

핵심 질문: 데모는 잘 되는데, 왜 프로덕션에 못 올리는가?

핵심 공식 — Agent = Model + Harness

날것의 LLM은 에이전트가 아니다. 하네스가 모델을 감싸야 비로소 에이전트가 된다.

Agent = Model + Harness

"지능"만으로는 부족하다 — 상태·도구·검증·제약이 결합되어야 동작하는 에이전트

MODEL

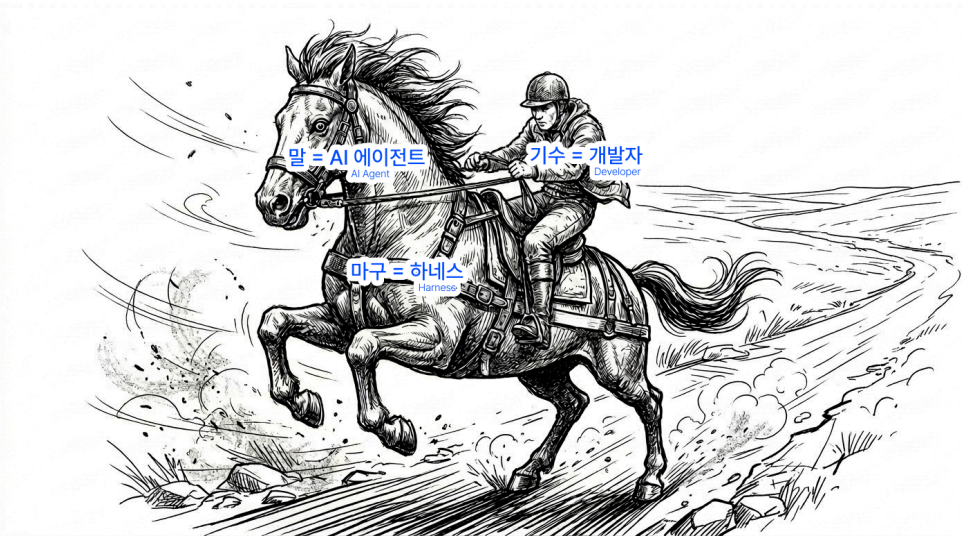
지능 (Intelligence)

LLM의 **추론·생성 능력** 자체. 같은 모델도 환경에 따라 성과가 크게 달라진다. 모델은 점점 **범용품**이 되어 간다 — Sonnet, GPT, Gemini 모두 비슷한 수준에 수렴.

HARNESS

제어 환경 (Control Environment)

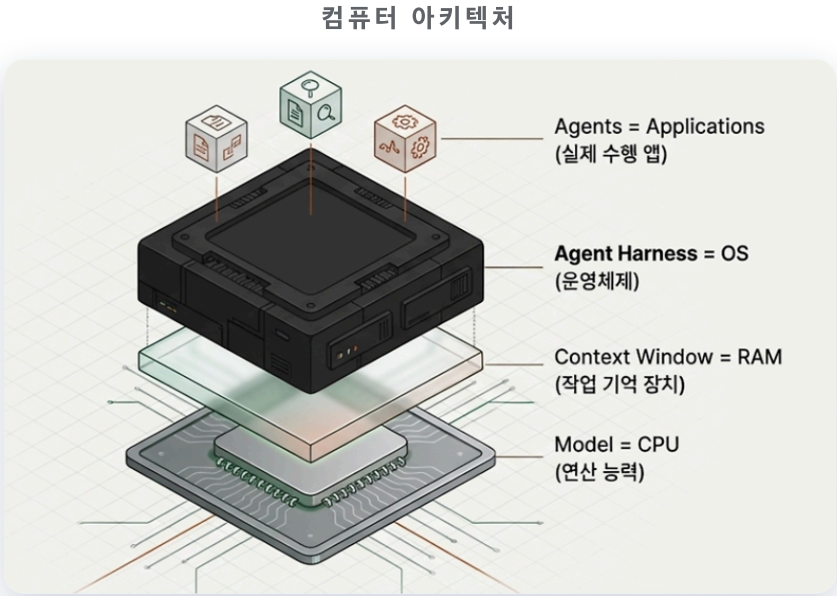
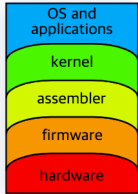
모델 주위의 **모든 장치** — 도구, 메모리, 컨텍스트, 검증, 제약, 복구 메커니즘까지. 말(馬)에 채우는 **고삐**와 **안장**의 비유.



"에이전트가 실수할 때마다, 그 실수를 두 번 다시 일으키지 않게 하는 장치를 설계한다."
— Mitchell Hashimoto (2026.2.5)

컴퓨터 아키텍처 비유

CPU(모델)가 강력해도 OS(하네스) 없이는 쓸모없다 — Philipp Schmid



≡



핵심: 같은 CPU도 OS가 다른 퍼포먼스가 달라진다 → 같은 LLM도 하네스가 다른 결과가 달라진다

하네스 6대 프리미티브 — 하네스는 무엇으로 구성되는가

"모델이 아닌 모든 것" · LangChain, The Anatomy of an Agent Harness

1

파일시스템 **FILESYSTEM**

가장 근본적인 프리미티브. 세션을 넘는 저장·상태, 컨텍스트 오프로드, 협업 표면. Git으로 버전·롤백.

2

코드 실행 **BASH / CODE**

고정된 도구 집합 대신 코드 실행. 모델이 필요한 도구를 즉석에서 만든다 — 자율 문제해결의 토대.

3

샌드박스 **SANDBOX**

격리된 안전 실행 환경. 런타임·CLI·브라우저 사전 설치, 테스트·로그로 셀프 검증, 온디맨드 확장.

4

메모리·검색 **MEMORY / SEARCH**

AGENTS.md로 지속 학습, 웹 검색·MCP로 지식 컷오프 너머의 최신 정보 접근.

5

컨텍스트 관리 **CONTEXT**

컴팩션, 도구 출력 오프로드, 스킬(점진적 공개)로 컨텍스트 로트를 막는다.

6

장기 실행 **LONG-HORIZON**

파일시스템+Git 멀티세션, Ralph Loop, 계획·셀프검증, 서브에이전트 병렬 위임.

핵심: 이 프리미티브들이 결국 "좋은 컨텍스트 엔지니어링"을 실제로 전달하는 수단이다

3개의 시대 — Prompt → Context → Harness

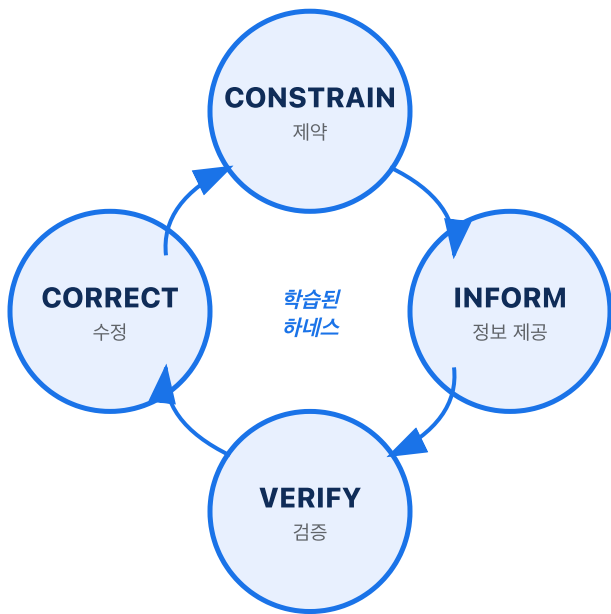
새 시대는 이전 시대를 대체하지 않고 흡수한다 (누적)

1ST ERA 프롬프트 엔지니어링 핵심 질문 "어떤 말을 쓸 것인가" 작업 단위 단일 API 호출 인간의 역할 프롬프트 작성자	2ND ERA 컨텍스트 엔지니어링 핵심 질문 "어떤 정보가 필요한가" 작업 단위 멀티턴 세션 인간의 역할 정보 아키텍트	3RD ERA · NOW 하네스 엔지니어링 핵심 질문 "어떤 환경이 필요한가" 작업 단위 완성된 기능 인간의 역할 환경 디자이너
--	--	---

하네스 엔지니어링 = **프롬프트 + 컨텍스트 + 환경 설계**를 모두 포함하는 상위 프레임워크

4대 기둥 — Constrain · Inform · Verify · Correct

독립적인 4개 영역이 아니라, 순환하는 사이클이다



제약 **CONSTRAIN**

에이전트가 할 수 있는 것을 제한 — 샌드박스, 도구 허용 목록, 비용 상한



정보 제공 **INFORM**

무엇을 해야 하는지 알린다 — AGENTS.md, MCP, 동적 컨텍스트



검증 **VERIFY**

올바르게 실행됐는지 확인 — Eval, 자동 테스트, 셀프 검증 루프



수정 **CORRECT**

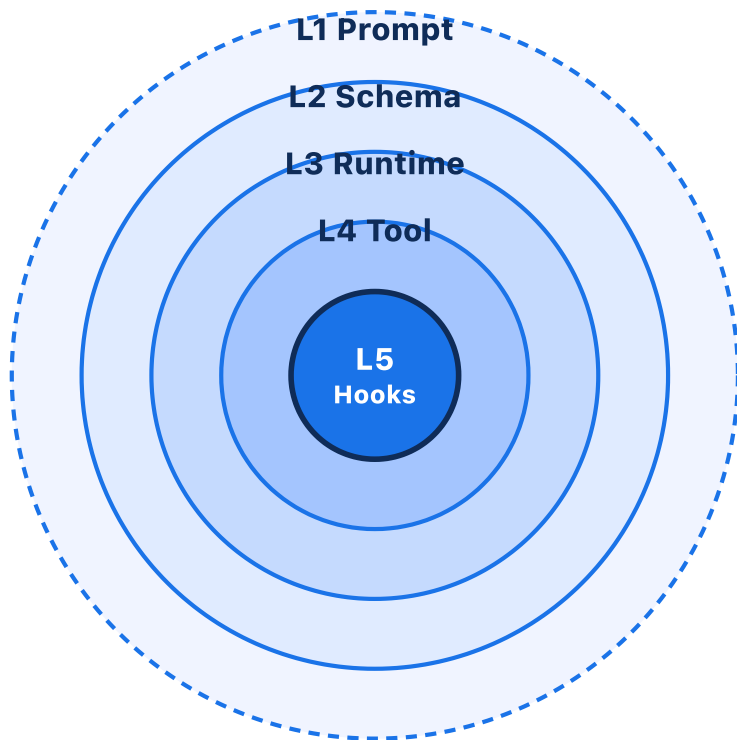
실수를 복구·학습 — 재시도, HITL, 가비지 컬렉션, 규칙 추가

순환의 본질: Correct에서 발견된 패턴이 Constrain의 새 규칙이 되고, Inform이 강화되고, Verify가 추가된다

Constrain — 제한이 자유보다 생산적이다

해공간(solution space)을 좁히면 정확도가 올라간다 · OpenDev 5계층 방어

5계층 방어 (Defense in Depth)



실전 — Claude Code .claude/settings.json

```
{
  "permissions": {
    "allow": [
      "Read", "Edit",
      "Bash(npm test)",
      "Bash(npm run lint)",
      "Bash(git *)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(curl *)",
      "Bash(sudo *)"
    ]
  }
}
```

Claude Code 퍼미션 3단계: Manual(매번 확인) → Semi-Auto(허용 목록만 자동) → Auto(YOLO 모드)

Inform — Goldilocks Zone

너무 구체적이지도, 너무 모호하지도 않은 적정 온도를 찾아라

TOO COLD



너무 구체적

상황 바뀌면 깨짐
유지보수 비용 폭증

JUST RIGHT ★



60줄의 황금률

강력한 휴리스틱 + 핵심 규칙
(Anthropic 권장)

TOO HOT



너무 모호

"잘 해줘" 수준
제각각 행동 / 일관성 X

Inform의 3가지 채널 — 정적 + 동적 + 이벤트

AGENTS.md

정적 문서 (규칙·맥락)



MCP / RAG

동적 컨텍스트 (실시간)



Reminder

이벤트 리마인더

핵심: 정보는 많을수록 좋은 게 아니다 — "필요한 것을, 필요할 때만"이 황금률

AGENTS.md — 에이전트의 학습된 지혜

실패 1건 = 방지책 1줄. 삭제하지 않고 누적한다.

실패 1건 = 방지책 1줄

— Hashimoto 원칙



매달 **6~8개** 규칙 추가
→ 1년이면 **50+개** 누적

- ▶ "~하지 마라"만 쓰지 말고 **"대신 ~해라"**까지 써라
- ▶ 한 달에 6~8개, 1년 50+개의 규칙이 자연스럽게 누적
- ▶ 새 에이전트가 투입돼도 **같은 실수를 반복하지 않는다**

⚠ ETH Zurich 연구: LLM이 자동 생성한 **AGENTS.md**는 성능 **3%** 하락, 비용 **20%** 증가 — 사람이 핵심을 골라야

실전 예시 — Ghostty 프로젝트

AGENTS.md — Ghostty Project

Rules

- Never modify files in src/core/ without permission
- Always run `zig build test` before committing
- Use snake_case for all function names

Past Failures → Prevention

2026-02-15: 메모리 누수 발견 후 추가

- **Do not use std.mem.alloc directly → use arena**

2026-03-02: UTF-8 파싱 버그 후 추가

- **Config parsing: validate UTF-8 before processing**

2026-03-18: 터미널 호환성 이슈 후 추가

- **Test escape sequences with all 6 major terminals**

Verify — Eval 주도 개발 (EDD)

AI 출력은 확률적 → 검증도 확률적 임계값으로 설계해야 한다

TDD vs EDD

TDD · 결정론적

assert output == expected

정확히 같아야 통과

EDD · 확률적

assert success_rate >= 0.8

10번 중 8번 이상이면 통과

ANTHROPIC 권장 임계값

보안 관련

95%+

코드 포맷·스타일

90%+

기능 구현

70~80%

Eval 기본 구조

```
def run_eval(task, agent, grader,
             threshold=0.8, runs=10):
    """에이전트 태스크 수행 능력 평가"""

    # N번 실행 후 점수 수집
    results = [
        grader.evaluate(agent.execute(task))
        for _ in range(runs)
    ]

    # 성공률 계산
    success_rate = sum(
        r >= 1.0 for r in results
    ) / len(results)

    return {
        'pass': success_rate >= threshold,
        'success_rate': success_rate
    }
```

Correct — Doom Loop vs Ralph Loop

AI는 반드시 실수한다. 중요한 건 복구 능력이다.

✗ ANTI-PATTERN

Doom Loop (둠 루프)

'doom' = 파멸. 경제·도시학의 "스스로 악화되는 하강 나선"에서 유래 → 오염된 컨텍스트를 끌고 재 시도하며 점점 더 망가지는 안티패턴

- 1 실패한 컨텍스트 그대로 재시도
- 2 같은 파일 10번 수정 시도
- 3 이전 실수가 새 시도를 오염
- 4 코드는 점점 더 망가진다

→ 더 큰 실패 / 비용 폭증

✓ PATTERN

Ralph Loop (랄프 루프)

Geoffrey Huntley(2025) 명명. 심슨 캐릭터 **Ralph Wiggum**에서 이름 차용 → "똑똑한 기억보다, 깨끗한 재시작이 더 강력하다"

- 1 실패 원인을 한 줄로 요약
- 2 깨끗한 컨텍스트 로 새 시도 시작
- 3 요약 + "다른 접근법을 시도하라"
- 4 오염 없이 N번 시도

→ 복구 가능 / 비용 통제

에스컬레이션 기준: 동일 에러 3회 · 비가역 작업(DELETE 등) · 비용 상한 도달 → **Human-in-the-Loop**로 전환

컨텍스트 관리 4대 전략

컨텍스트는 희소 자원 — "덜 넣되 정확히 넣는 것"이 본질

STRATEGY 01



JIT 검색 (Just-in-Time)

사전에 모든 정보를 로드하지 않고, **필요한 순간에만 검색**해서 컨텍스트에 추가

사전 로딩 대비 70% 토큰 절약

STRATEGY 02



하이브리드 전략

자주 쓰는 핵심은 **사전 로딩**, 그 외는 **자율 탐색**으로 동적 인출

사전 로딩 + JIT의 균형점

STRATEGY 03



컴팩션 5단계

요약 → 트렁케이션 → 버킷팅 → 메모리 병합 → 퇴거 — 컨텍스트 윈도우가 가득 차기 전 단계적 압축

한계 도달 전 자동 정리

STRATEGY 04



서브에이전트 = 컨텍스트 방화벽

하위 작업을 격리된 컨텍스트에서 처리 후 **요약만 메인 컨텍스트로** 반환

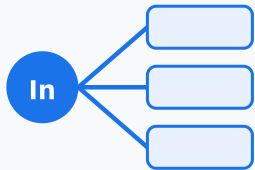
50,000 토큰 → 200 토큰 요약

5대 오케스트레이션 패턴

단순한 패턴부터 시작 → 측정 결과가 한계를 보일 때만 복잡도 추가

PATTERN 01

Router



입력 → 분류 → 적절한 핸들러로 분배

예) 고객 문의 분류
환불·기술 자동 분기

● ● ● ● ● 단순

PATTERN 02

Chain



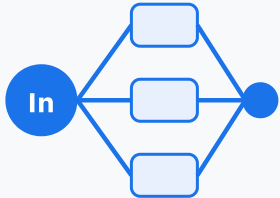
A → B → C 순차 실행 (파이프라인)

예) 번역 → 요약 → 키워드
단계별 NLP 파이프라인

● ● ● ● ● 매우 단순

PATTERN 03

Parallel



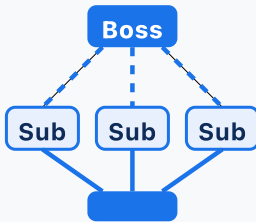
A + B + C 동시 실행 → 결과 통합

예) 다중 LLM 답변 비교
독립 분석 동시 수행

● ● ● ● ● 중간

PATTERN 04

Orchestrator



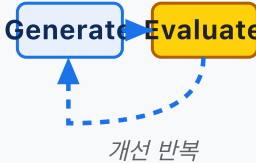
상위 위임 → 하위 실행 → 요약 회신

예) 복잡한 리서치
코드베이스 탐색·요약

● ● ● ● ● 복잡

PATTERN 05

Evaluator-Optimizer



생성 → 평가 → 개선 반복 (셀프 비평)

예) 글쓰기·코드 리뷰
품질 반복 개선

● ● ● ● ● 매우 복잡

☑ 언제 이 패턴을 쓸까? — 측정 결과가 한계를 보일 때만 다음 단계로

ROUTER

입력 종류가
뚜렷해질 때

CHAIN

단계 순서가
명확할 때

PARALLEL

독립 작업이
3개 이상일 때

ORCHESTRATOR

작업이 너무
크고 막연할 때

EVALUATOR-OPT

품질 이슈가
Eval로 측정될 때

4대 플랫폼 비교 — 어떤 하네스를 쓰는가

"최고의 플랫폼"은 없다 — 보안·팀 규모·워크플로에 따라 선택



Claude Code

ANTHROPIC

- ▶ 퍼미션 3단계 (Manual/Semi/Auto)
- ▶ Hooks로 위험 명령 차단
- ▶ 3층 정보: CLAUDE.md → Skills → MCP
- ▶ Plan 모드 (읽기 전용 계획)



OpenAI Codex

OPENAI

- ▶ Firecracker 마이크로VM 완전 격리
- ▶ 커널 수준 보안 (Linux KVM)
- ▶ 병렬 다중 작업 지원
- ▶ 100만 줄 무인 머지 사례



Cursor

ANYSPHERE

- ▶ 모델별 튜닝된 기본 설정
- ▶ MDC Rules (경로별 조건부 규칙)
- ▶ IDE 통합 우선 (Composer)
- ▶ 인라인 자동완성 강제



Devin

COGNITION

- ▶ Brain + Workspace 분리 아키텍처
- ▶ 계획 ↔ 실행 분리
- ▶ Slack/JIRA 통합 (티켓 → 코드)
- ▶ 장기 실행 작업 특화

실패하는 하네스의 7가지 징후

하네스는 코드보다 빠르게 썩는다 — 3개 이상 "예"면 쇠퇴 경로



DESIGN 설계 안티패턴 4종

1 과도한 제약
에이전트가 아무 일도 못 하는 수준의 제한
✓ 해결: 점진적 퍼미션 완화 · Plan 모드 활용

2 규칙 폭발
500줄짜리 AGENTS.md → 모델이 무시하기 시작
✓ 해결: 60줄 황금률 · 핵심 규칙만 최상단

3 Eval 부재
"잘 되는 것 같다"로 판단 — 측정 없는 운영
✓ 해결: 회귀 테스트 · 주간 eval 자동화

4 모델 의존 착각
"모델만 좋으면 다 해결" — 하네스 무시
✓ 해결: 4기동(C-I-V-C) 균형 투자



OPS 운영 안티패턴 3종

5 하네스 부패
문서와 실제 코드가 어긋남 → 거짓 정보 학습
✓ 해결: 분기별 하네스 감사(audit)

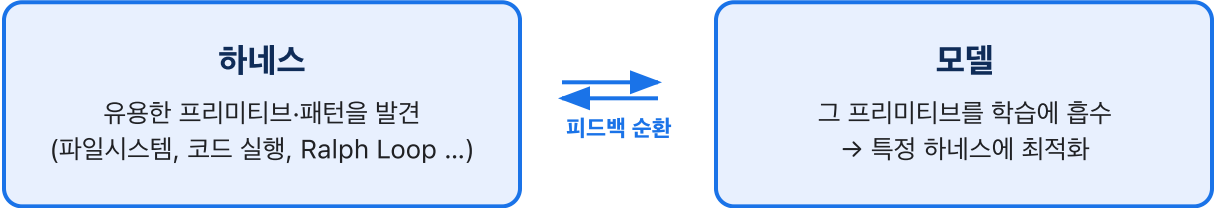
6 비용 블랙홀
월 \$200 → \$2,000 → \$20,000... 누가 모니터링하나?
✓ 해결: 비용 상한 · 알림 자동화

7 거버넌스 부재
누가 AGENTS.md를 수정하는가? 리뷰 프로세스는?
✓ 해결: 오퍼십 지정 · PR 리뷰 필수화

자가진단: 7개 중 3개 이상 "예"면 하네스가 쇠퇴 경로에 들어선 것

모델-하네스 공진화 — 하네스가 모델 학습의 일부가 된다

모델이 하네스를 "in the loop"로 함께 학습하는 시대



양날의 검
모델이 특정 하네스에 최적화되면 표적 성능은 오르지만, 일반화는 어려워지는 트레이드오프가 생긴다.

그래도 하네스는 남는다
일부 책임은 모델 안으로 흡수되겠지만, 잘 갖춰진 환경·상태·검증 루프는 어떤 모델이든 더 낮게 만든다.

월요일 아침 — 3가지 액션

거창한 인프라가 아니라, 프로젝트 루트의 마크다운 파일 하나에서 시작

1

5분

AGENTS.md 생성

빈 템플릿 복사 → 프로젝트 루트에 저장. 그게 시작이다.

2

매번

실수 1건 = 규칙 1줄

에이전트가 실수할 때마다 **방지 규칙 1줄**을 누적해라

3

주간

측정하라

테스트 통과율 · 토큰 비용 · 작업 시간 — 숫자가 진실

모델은 **범용품**이다.
하네스가 **해자**다.

— Harness Engineering, 2026

C

Constrain

I

Inform

V

Verify

C

Correct



해자 (壕子, moat)

壕 가장자리 해 · 子 아들 자 (둘러싼 것의 뜻)

① **원뜻**: 성을 둘러싼 방어용 수로 — 적의 침입을 물리적으로 차단

② **비유**: Warren Buffett이 강조한 **economic moat** — 경쟁사가 쉽게 넘볼 수 없는 구조적 우위